

Vor- und Nachteile der Test-getriebenen Entwicklung

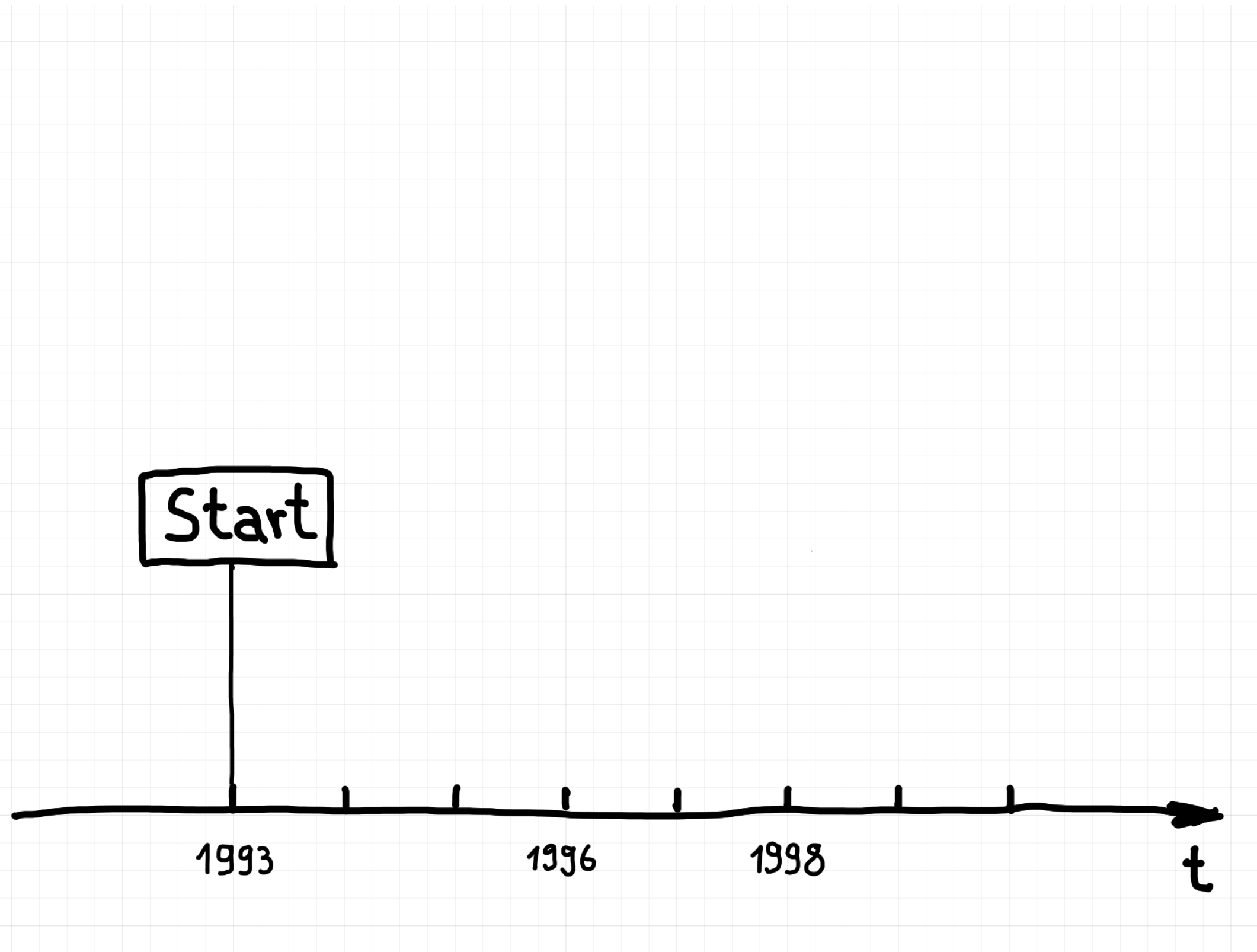
Dominik Hauser

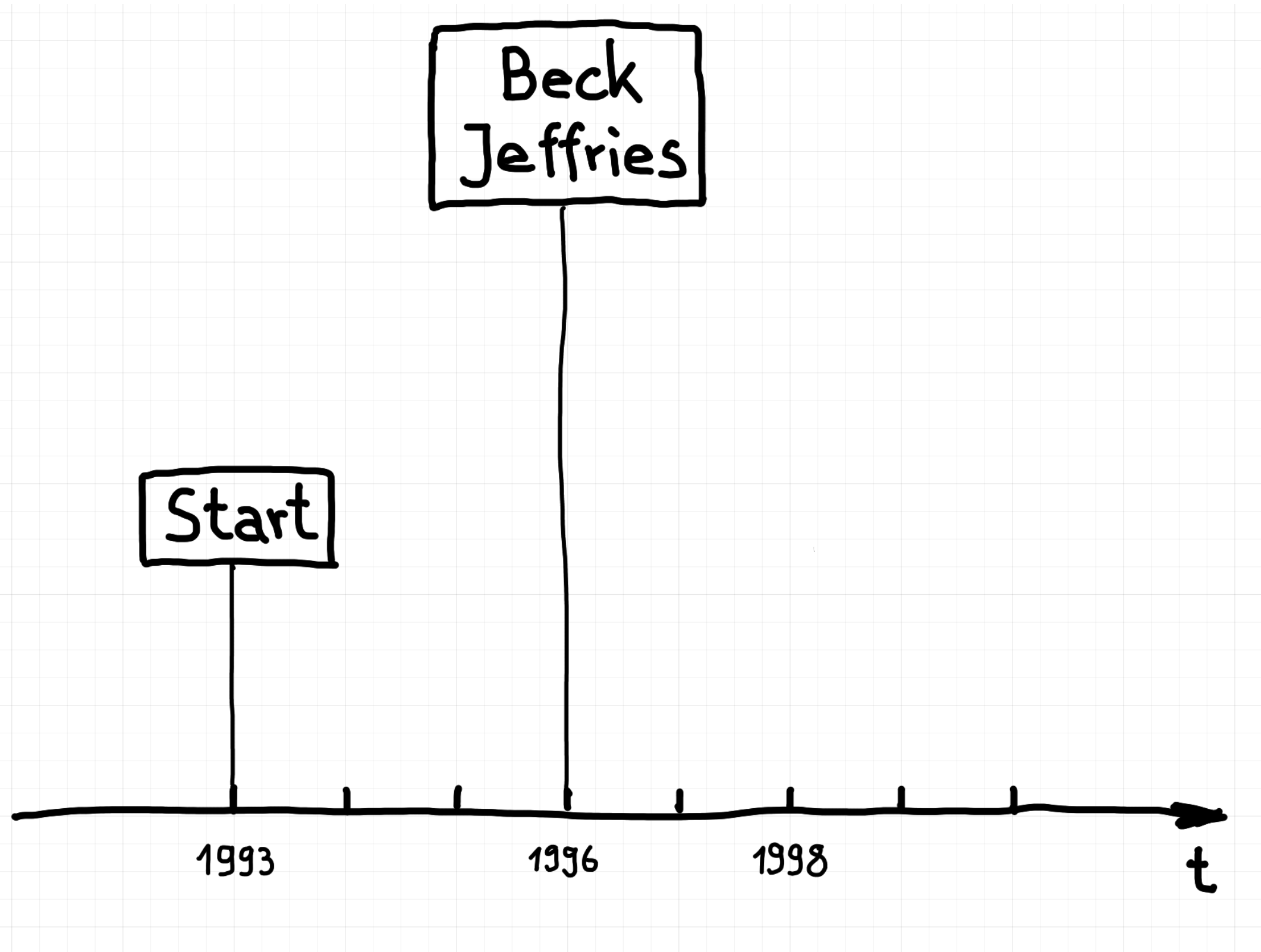
iOS- und macOS-Entwickler

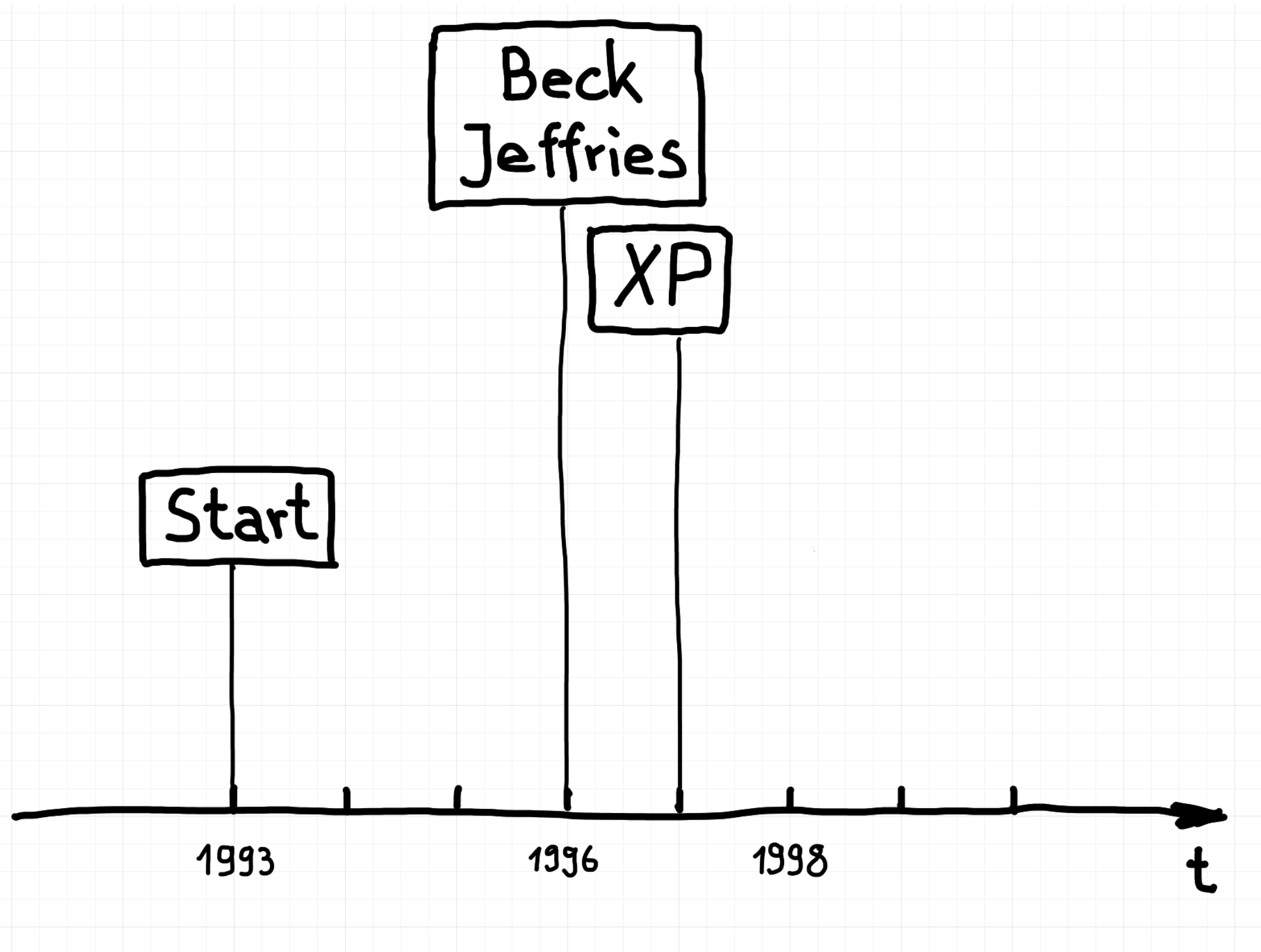
Das C3-Projekt¹

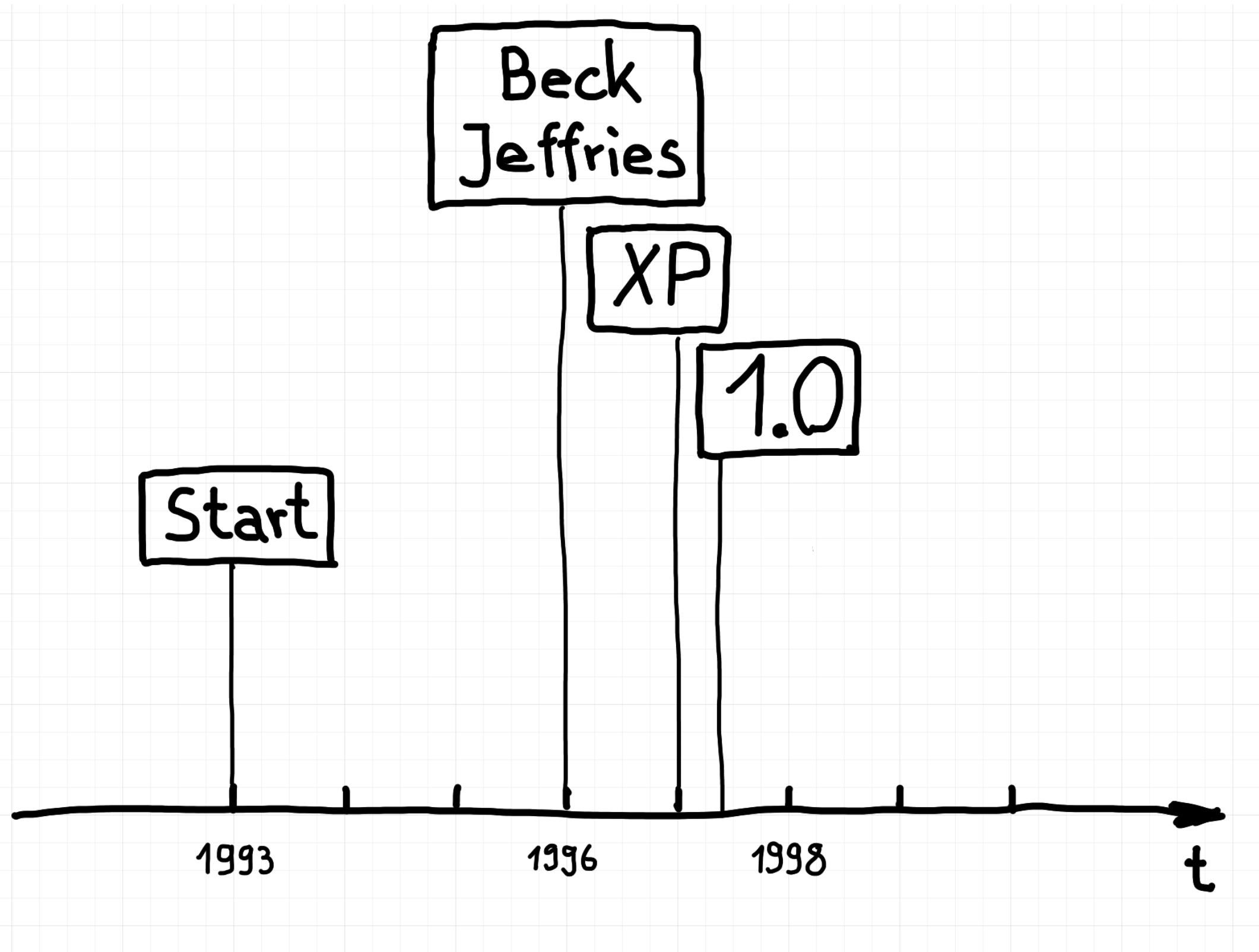
- Chrysler Comprehensive Compensation System
- Smalltalk und GemStone
- Ziel: Gehaltsabrechnungen für 87.000 Mitarbeiter bis 1999

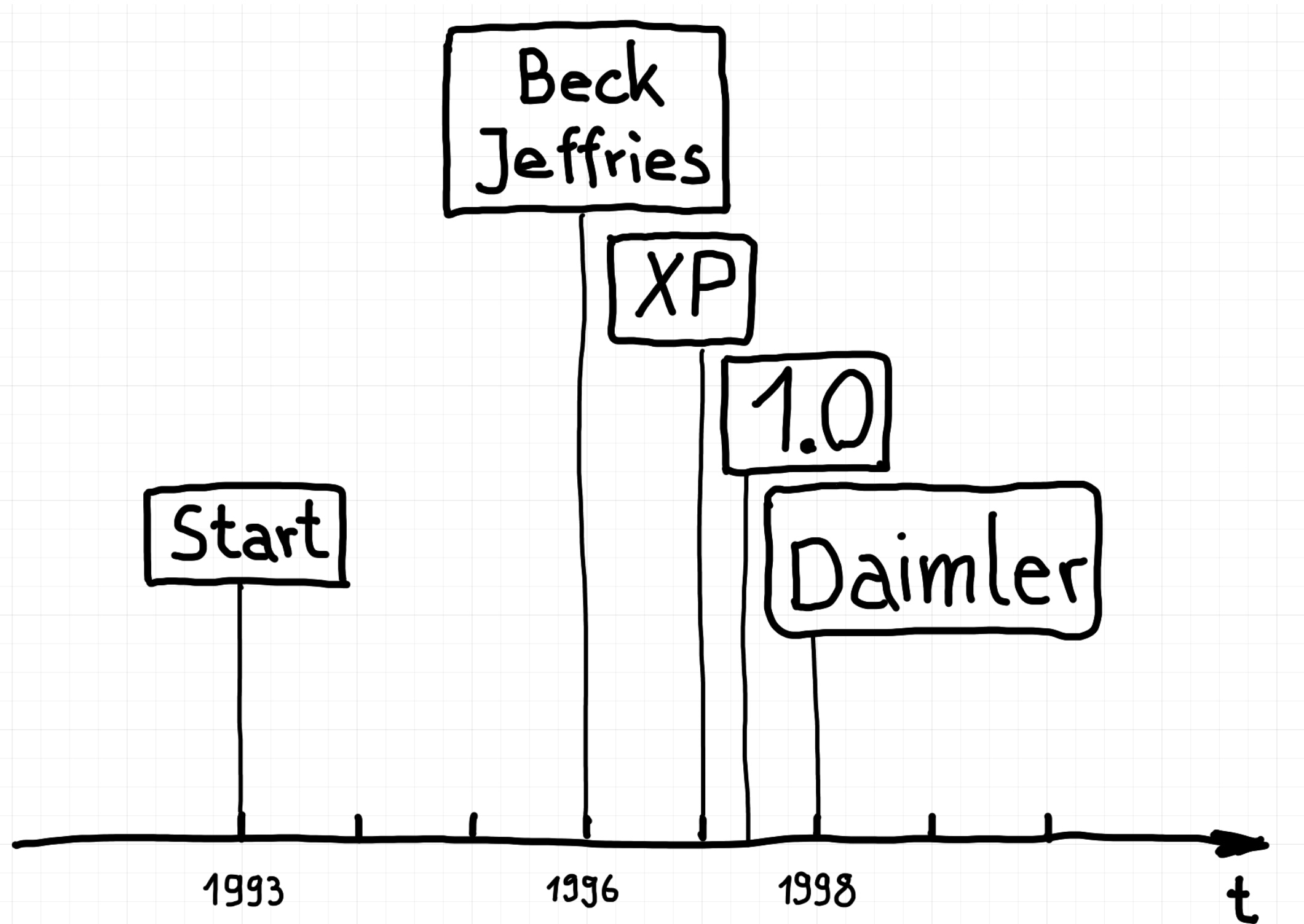
¹ Quelle [Wikipedia](#).

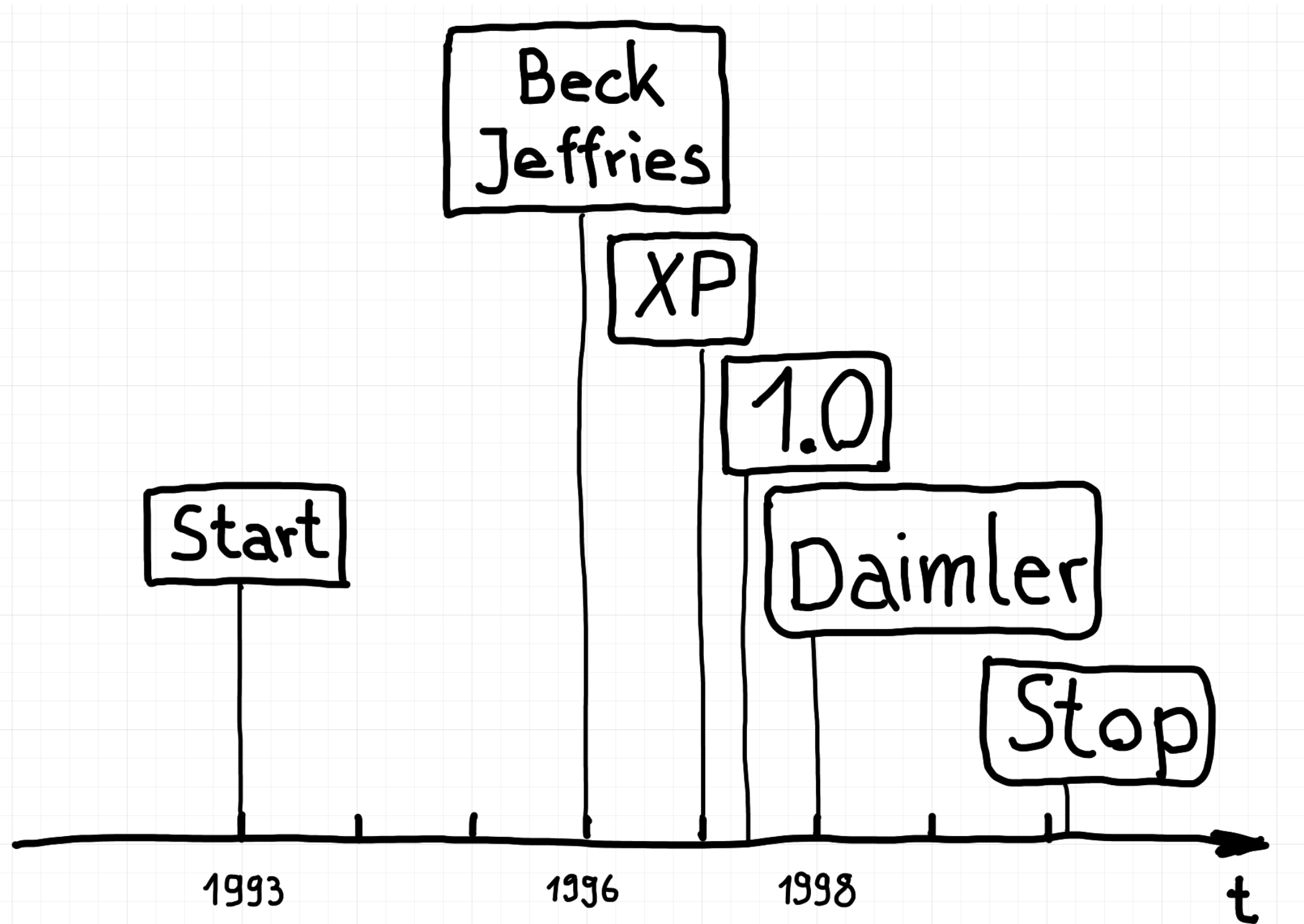












Regeln des Extreme Programming (Auszug)

Coding (Auszug)

- Unit-Test zuerst
- Pair-Programming
- Integriere oft
- Dedizierter Integrationscomputer

Testing (Auszug)

- Aller Code muss Unit-Tests haben
- Aller Code muss die Tests passieren vor den Release
- Bug -> Tests

Regeln des Extreme Programming (Auszug)

Coding (Auszug)

- **Unit-Test zuerst**
- Pair-Programming
- Integriere oft
- Dedizierter Integrationscomputer

Testing (Auszug)

- Aller Code muss Unit-Tests haben
- Aller Code muss die Tests passieren vor den Release
- Bug -> Tests



The Rules of Extreme Programming

Planning

- User stories are written.
- Release planning creates the release schedule.
- Make frequent small releases.
- The project is divided into iterations.
- Iteration planning starts each iteration.

Managing

- Give the team a dedicated open work space.
- Set a sustainable pace.
- A stand up meeting starts each day.
- The Project Velocity is measured.
- Move people around.
- Fix XP when it breaks.

Designing

- Simplicity.
- Choose a system metaphor.
- Use CRC cards for design sessions.
- Create spike solutions to reduce risk.
- No functionality is added early.
- Refactor whenever and wherever possible.



Coding

- The customer is always available.
- Code must be written to agreed standards.
- Code the unit test first.
- All production code is pair programmed.
- Only one pair integrates code at a time.
- Integrate often.
- Set up a dedicated integration computer.
- Use collective ownership.

Testing

- All code must have unit tests.
- All code must pass all unit tests before it can be released.
- When a bug is found tests are created.
- Acceptance tests are run often and the score is published.

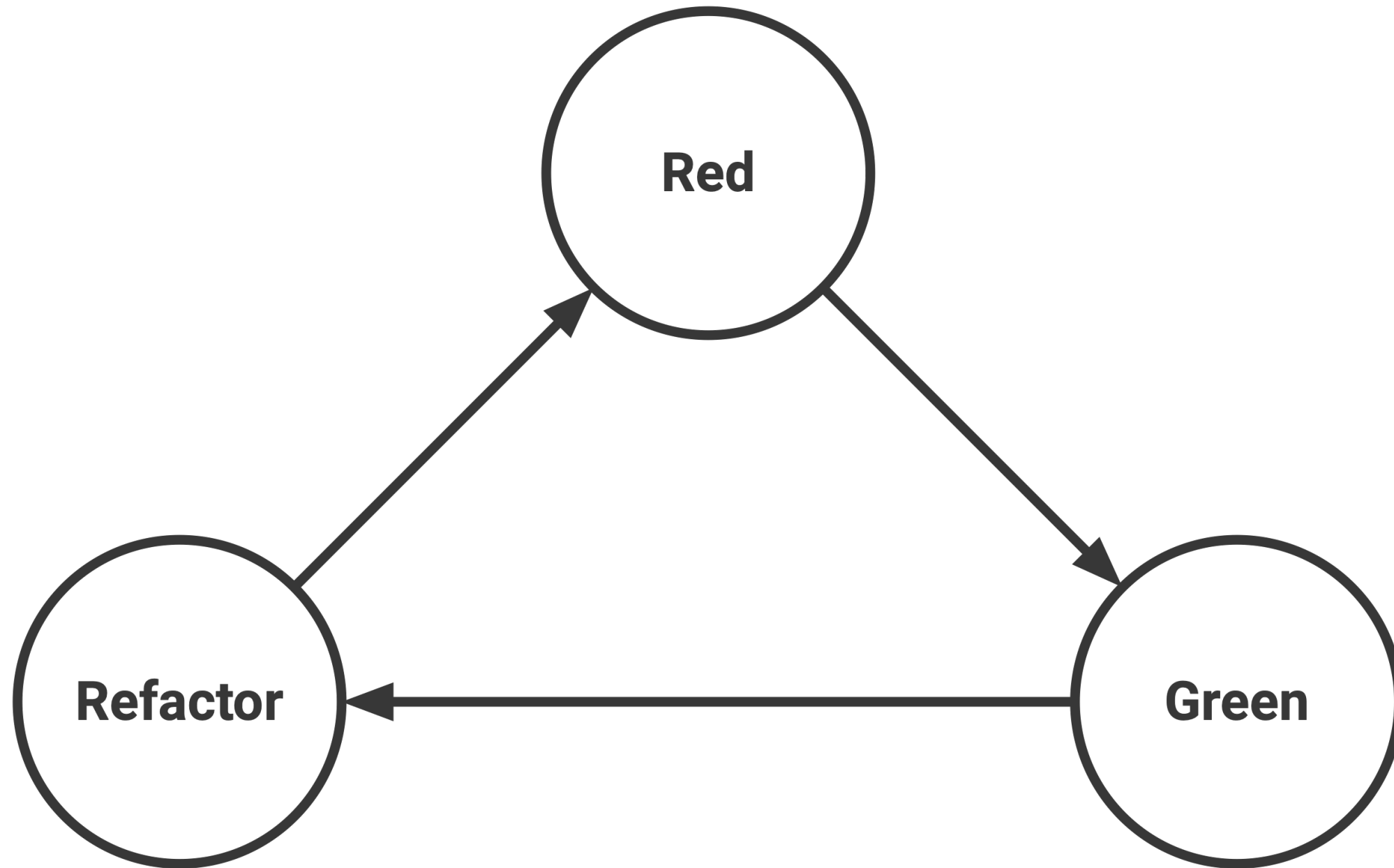
Let's review the values of Extreme Programming (XP) next. ⌂ 🔍 📄

[ExtremeProgramming.org home](#) | [XP Map](#) | [XP Values](#) | [Test framework](#) | [About the Author](#)

Copyright 1999 Don Wells all rights reserved

Quelle: <http://www.extremeprogramming.org/rules.html>

Test Driven Development



Red

Schreibe einen Test, der fehlschlägt

- Kompilierfehler -> Testfehler
- Test muss *echt* fehlschlagen
- Sobald der Test fehlschlägt sollte Produktivcode geschrieben werden

Green

Schreibe gerade so viel Code, dass Test durchläuft

- Schreibe den simpelsten Code
- You Ain't Gonna Need It (YAGNI)

Refactor

Verbessere den Produktiv- und den Test-Code

- Test-Code ist auch Code
- Tests helfen dabei nichts kaputt zu machen

Simple Beispi

Eine Wallet, in die die Nutzerin einzahlen kann.

Quelle: <https://quii.gitbook.io/learn-go-with-tests>

Simple Beispiel (Red)

wallet_test.go

```
func TestWallet(t *testing.T) {  
    wallet := Wallet{}  
    wallet.Deposit(10)  
  
    got := wallet.Balance()  
    want := 10  
  
    if got != want {  
        t.Errorf("got %d want %d", got, want)  
    }  
}
```

Simple Beispi (Red)

```
$> go test
# example.com/hello/wallet [example.com/hello/wallet.test]
./wallet_test.go:6:12: undefined: Wallet
FAIL    example.com/hello/wallet [build failed]
```

Simple Beispiel (Green)

wallet.go

```
type Wallet struct {  
  
}
```

Simple Beispiel (Green)

```
$> go test
# example.com/hello/wallet [example.com/hello/wallet.test]
./wallet_test.go:7:9: wallet.Deposit undefined
    (type Wallet has no field or method Deposit)
./wallet_test.go:9:16: wallet.Balance undefined
    (type Wallet has no field or method Balance)
FAIL    example.com/hello/wallet [build failed]
```

Simple Beispiel (Green)

wallet.go

```
type Wallet struct {  
  
}  
  
func (w Wallet) Deposit(amount int) {  
  
}  
  
func (w Wallet) Balance() int {  
    return 0  
}
```

Simple Beispiel (Green)

```
$> go test
--- FAIL: TestWallet (0.00s)
    wallet_test.go:13: got 0 want 10
FAIL
exit status 1
FAIL    example.com/hello/wallet    0.377s
```

Simple Beispiel (Green)

wallet.go

```
type Wallet struct {  
    balance int  
}
```

```
func (w Wallet) Deposit(amount int) {  
    w.balance += amount  
}
```

```
func (w Wallet) Balance() int {  
    return w.balance  
}
```

Simple Beispiel (Green)

```
go test
```

```
--- FAIL: TestWallet (0.00s)
```

```
    wallet_test.go:13: got 0 want 10
```

```
FAIL
```

```
exit status 1
```

```
FAIL    example.com/hello/wallet    0.404s
```

Simple Beispiel (Green)

wallet.go

```
type Wallet struct {  
    balance int  
}  
  
func (w *Wallet) Deposit(amount int) {  
    w.balance += amount  
}  
  
func (w *Wallet) Balance() int {  
    return w.balance  
}
```

Simple Beispiel (Green)

```
$> go test
```

```
PASS
```

```
ok      example.com/hello/wallet    0.263s
```

Simple Beispiel (Refactor)

Nichts zu tun



TDD - Nachteile

- (Zunächst) langsamer
- Lernkurve
- Alle im Team müssen mitziehen
- Rechtfertigung (am Anfang)
- Druck ("Mach mal eben")
- Disziplin

TDD - Vorteile

- Refaktorierbarkeit (Agilität)
- Schnelles Feedback
- Dokumentation
- Gedankenstütze
- Regressionsschutz
- Doppelte Buchführung

Eigenschaften guter Unit-Tests²

- Sensitiv: Verhalten
- Insensitiv: Struktur
- Lesbar
- Schreibbar
- Schnell
- Deterministisch
- Automatisiert
- Isoliert
- Komponierbar
- Spezifisch
- Vorhersagend
- Inspirierend

² Quelle: <https://tidyfirst.substack.com/p/desirable-unit-tests>

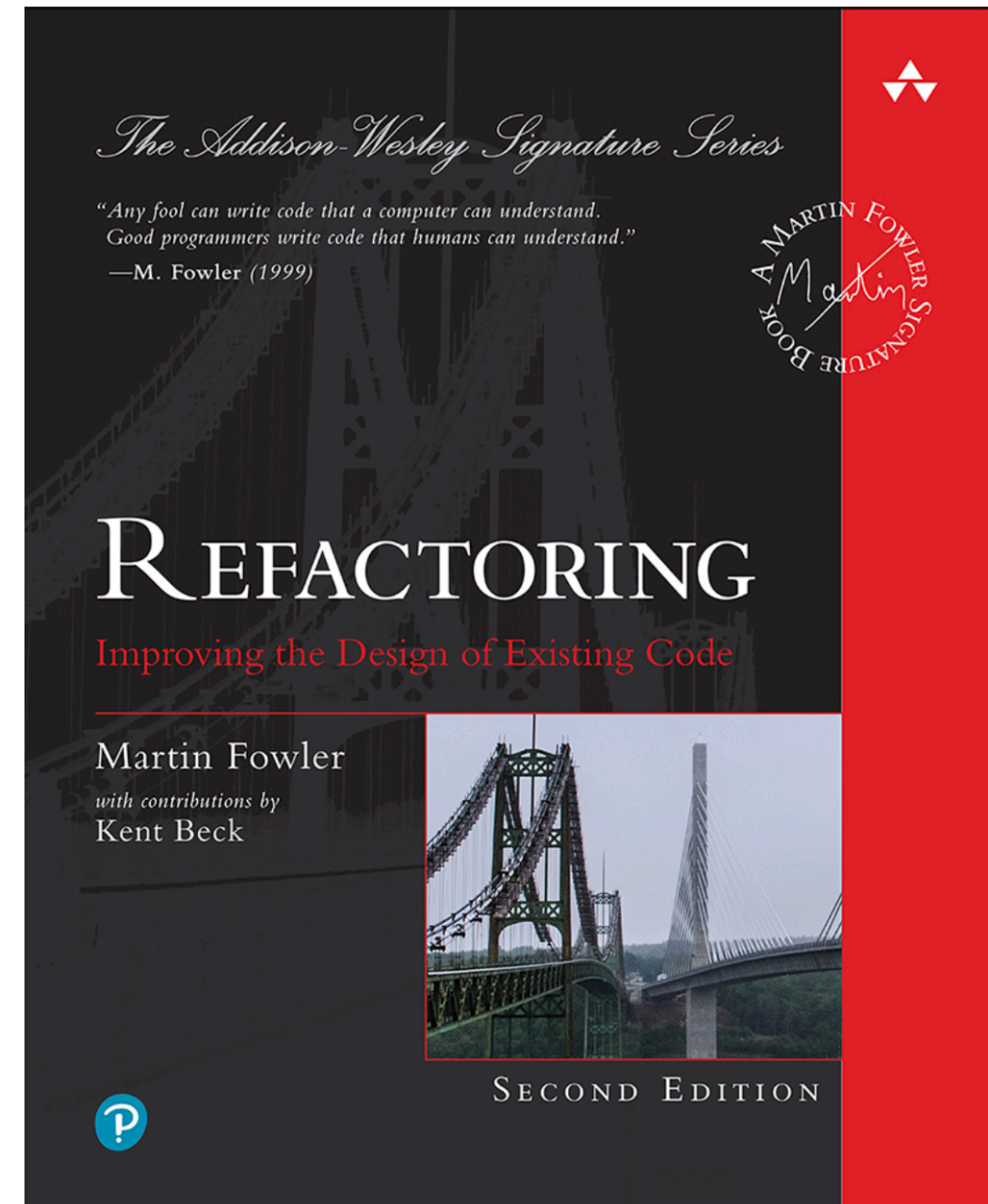
Test Doubles³

- **Dummy**: Platzhalter
- **Fake**: spielt dem Test Funktionen vor; nutzt Abkürzungen (In-Memory-DB...)
- **Stub**: liefert kontrollierte Antworten für den Test (API-Calls...)
- **Spy**: ein Stub, das auch Anfragen aufzeichnet
- **Mock**: vorprogrammiert mit Erwartungen

³ Quelle: <https://www.martinfowler.com/articles/mocksArentStubs.html>

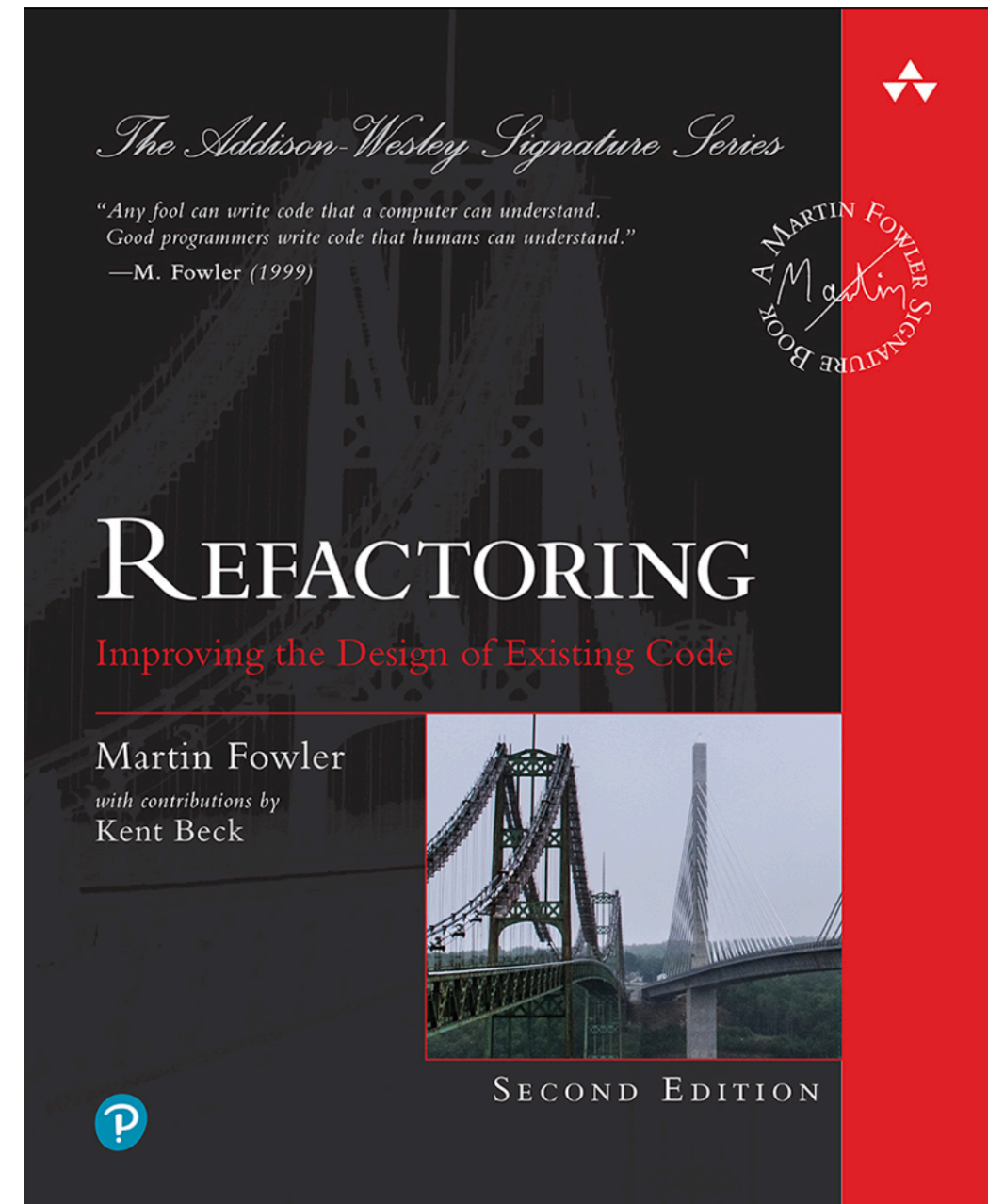
Refactoring

"Before you start refactoring, make sure you have a solid suite of tests. These tests must be self-checking."



Refactoring

"Although it takes time to build the tests, I end up saving that time, with considerable interest, by spending less time debugging."

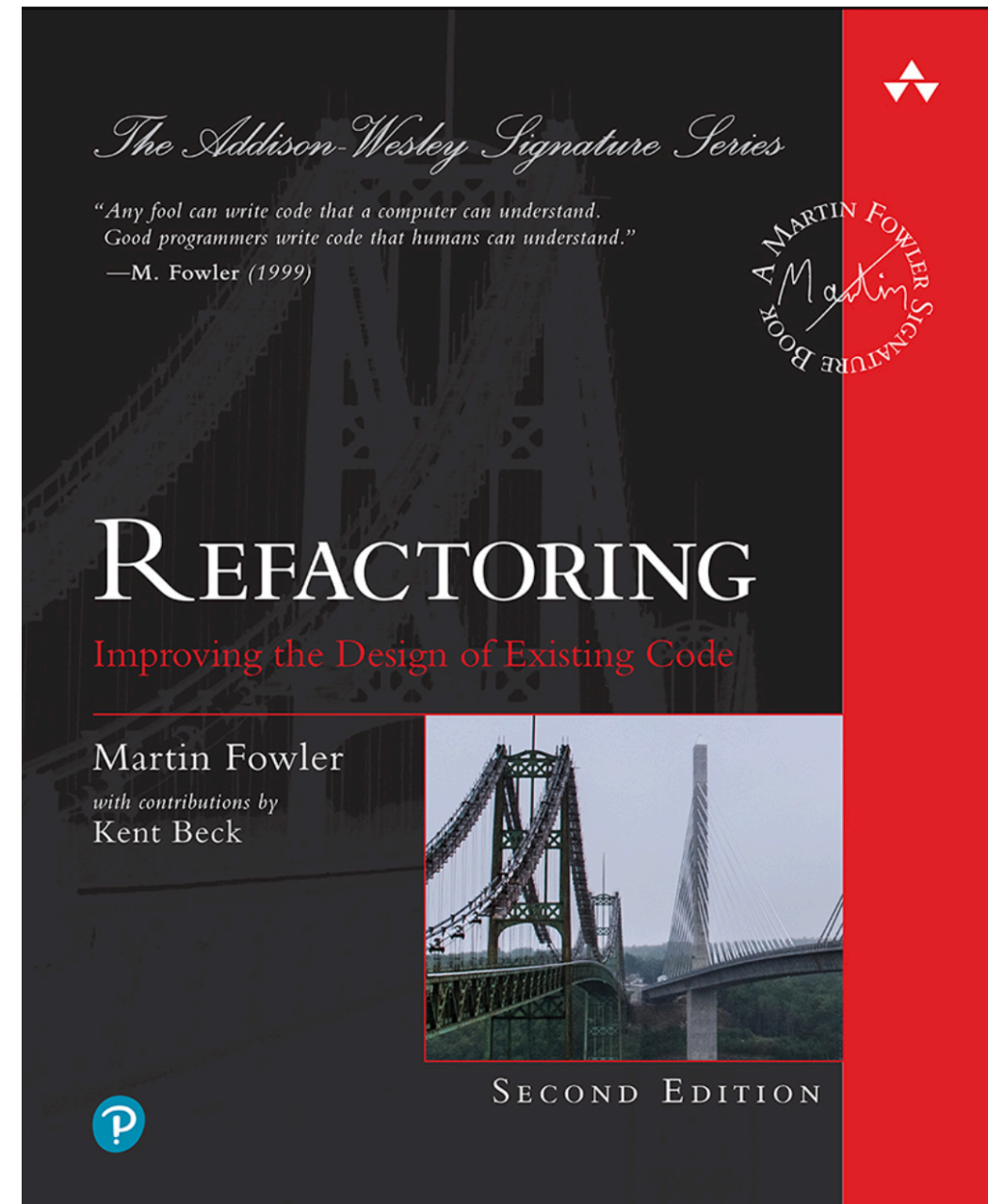


Refactoring

List of Refactorings

Change Function Declaration (124)
Change Reference to Value (252)
Change Value to Reference (256)
Collapse Hierarchy (380)
Combine Functions into Class (144)
Combine Functions into Transform (149)
Consolidate Conditional Expression (263)
Decompose Conditional (260)
Encapsulate Collection (170)
Encapsulate Record (162)
Encapsulate Variable (132)
Extract Class (182)
Extract Function (106)
Extract Superclass (375)
Extract Variable (119)
Hide Delegate (189)
Inline Class (186)
Inline Function (115)
Inline Variable (123)
Introduce Assertion (302)
Introduce Parameter Object (140)
Introduce Special Case (289)
Move Field (207)
Move Function (198)
Move Statements into Function (213)
Move Statements to Callers (217)
Parameterize Function (310)
Preserve Whole Object (319)
Pull Up Constructor Body (355)

Pull Up Field (353)
Pull Up Method (350)
Push Down Field (361)
Push Down Method (359)
Remove Dead Code (237)
Remove Flag Argument (314)
Remove Middle Man (192)
Remove Setting Method (331)
Remove Subclass (369)
Rename Field (244)
Rename Variable (137)
Replace Command with Function (344)
Replace Conditional with Polymorphism (272)
Replace Constructor with Factory Function (334)
Replace Derived Variable with Query (248)
Replace Function with Command (337)
Replace Inline Code with Function Call (222)
Replace Loop with Pipeline (231)
Replace Nested Conditional with Guard Clauses (266)
Replace Parameter with Query (324)
Replace Primitive with Object (174)
Replace Query with Parameter (327)
Replace Subclass with Delegate (381)
Replace Superclass with Delegate (399)
Replace Temp with Query (178)
Replace Type Code with Subclasses (362)
Separate Query from Modifier (306)
Slide Statements (223)
Split Loop (227)
Split Phase (154)
Split Variable (240)
Substitute Algorithm (195)



Wie starten?

- Neues Feature
- Bug -> Test
- Dokumentation
- Einfach anfangen :)

Changes



Quelle: <https://x.com/KentBeck/status/250733358307500032>

test && commit || revert⁴

Oddmund Strømme, the first programmer I've found as obsessed with symmetry as I am, suggested that if the tests failed the code should be reverted. I hated the idea so I had to try it.

-- Kent Beck

⁴ Quelle: https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864

Vielen Dank!

Fragen?